

Servizi Angular: GuestService, Observable e Dependency Injection

Il `GuestService` è un servizio Angular decorato con `@Injectable({ providedIn: 'root' })`, il che significa che viene istanziato una sola volta e reso disponibile in tutta l'applicazione tramite il sistema di Dependency Injection. Al suo interno, `HttpClient` viene iniettato tramite la funzione `inject()`, che rappresenta il nuovo approccio Angular per ottenere dipendenze senza ricorrere al costruttore: la funzione recupera l'istanza del servizio dal contesto di Dependency Injection e la assegna direttamente come proprietà della classe.

Il servizio espone due metodi pubblici, entrambi con tipo di ritorno `Observable<Guest>`. Il metodo `findBySsn` costruisce dinamicamente l'URL concatenando la base `apiURL` con il percorso `/byssn/` e il valore del parametro ricevuto, dopodiché esegue una richiesta GET tramite `this.http.get<Guest>(url)`. Il metodo `insert` esegue invece una richiesta POST tramite `this.http.post<Guest>()`, accettando due argomenti: il primo è `this.apiURL`, la stringa dell'endpoint a cui viene inviata la richiesta; il secondo è `guest`, ovvero il corpo della richiesta, che Angular serializza automaticamente in formato JSON prima dell'invio. Entrambi i metodi restituiscono un `Observable` perché le operazioni HTTP sono asincrone: nel caso dell'inserimento, l'`Observable` serve a notificare al chiamante se l'operazione è andata a buon fine o meno.

È fondamentale ricordare che un `Observable` è cold per natura: non esegue alcuna operazione finché non esiste almeno un subscriber. Senza una chiamata a `.subscribe()`, la richiesta HTTP non viene mai avviata.

Il Template HTML e i Binding sul Form

Il template del componente contiene un `<form>` con un campo di input per il codice fiscale. Su questo campo convergono diversi meccanismi Angular. L'attributo `name="ssn"` è obbligatorio quando si utilizza `ngModel` in un form Angular: identifica il controllo all'interno del form e consente ad Angular di registrarlo nell'oggetto `NgForm`. Il binding `[(ngModel)]="ssn"` utilizza la cosiddetta sintassi banana-in-a-box, che combina il property binding e l'event binding in un'unica direttiva: quando l'utente digita nel campo, la proprietà `ssn` nel componente si aggiorna automaticamente, e viceversa.

L'evento `(blur)="checkPresent()"` si attiva nel momento in cui l'utente abbandona il campo, ad esempio cliccando altrove o premendo Tab, e invoca il metodo `checkPresent()` nel componente. Questo approccio garantisce che la validazione avvenga solo dopo che l'utente ha terminato di digitare, evitando chiamate HTTP ad ogni singola pressione di tasto. L'evento `(keypress.enter)="checkPresent()"` è una sintassi di event binding con key modifier che permette di ascoltare specificatamente la pressione del tasto Invio, senza dover gestire manualmente il codice del tasto.

Il Componente GuestCheckIn

Nel componente `GuestCheckIn`, `GuestService` viene iniettato tramite `inject(GuestService)`, assegnandolo direttamente come proprietà della classe. La proprietà `ssn` è dichiarata come `signal<string>("")`, ovvero un Signal Angular: un valore reattivo che notifica automaticamente il framework quando cambia. Il tipo generico `<string>` indica il tipo del valore contenuto, mentre la stringa vuota è il valore iniziale. Per leggere il valore di un signal si invoca la proprietà come funzione, scrivendo `this.ssn()`.

Il metodo `checkPresent()` ha tipo di ritorno `void` e si apre con una guard clause: `if (!this.ssn()) return`. Se il campo SSN è vuoto, la stringa vuota è falsy in JavaScript, quindi il metodo esce immediatamente senza effettuare alcuna chiamata HTTP. Questo pattern, detto `early return`, evita richieste inutili al backend. Se invece il valore è presente, viene invocato `this.guestService.findBySsn(this.ssn())`, passando il valore corrente del signal, e sul risultante `Observable` viene chiamato `.subscribe()` con la sintassi a oggetto `Observer`. La callback `next` viene eseguita quando la risposta arriva con successo, ricevendo il JSON restituito dal server; la callback `error` viene eseguita se la chiamata HTTP fallisce, ad esempio con un 404 o un 500.

CORS e Same-Origin Policy

La Same-Origin Policy è una regola di sicurezza applicata dai browser per cui una pagina web può effettuare richieste HTTP solo verso la stessa origine da cui è stata caricata. Per "origine" si intende la combinazione di protocollo, dominio e porta: anche se il dominio è identico, una porta diversa è considerata un'origine differente. In un'applicazione Angular tipica il frontend gira su `localhost:4200` e il backend su `localhost:8080`: nonostante condividano lo stesso dominio, il browser blocca le richieste perché le porte sono diverse.

CORS, che sta per Cross-Origin Resource Sharing, è il meccanismo basato su header HTTP che risolve questo problema. Non è il browser a decidere autonomamente di permettere le richieste cross-origin: è il server che, tramite appositi header di risposta, comunica al browser quali origini esterne sono autorizzate ad accedervi. In sostanza, abilitare CORS sul backend significa dichiarare esplicitamente che si accettano richieste provenienti da origini diverse, incluse quelle su porte differenti dello stesso host.